

# **Computational Vision**

## **Motion Analysis Mid-term Assignment**

Change Detection and Blob Matching

**Mojtaba Alehosseini**

Student number: 5965588

May 2026

## 1. Objective and scope

I built a change detection and blob matching pipeline for the motion analysis mid-term and tested it on the provided station video plus three videos that I recorded myself. The pipeline follows the version one and version two algorithms from the motion analysis lectures, with one optional refinement (selective background update) and one optional extension (persistent track IDs that integrate matches across longer time spans). I also evaluated Otsu thresholding from the segmentation lecture as an alternative threshold rule and reported on its applicability to change detection.

This report mirrors the notebook section by section. Every claim about behaviour ties back to a number printed by the notebook or to a figure produced inside it. The figures in this report are taken directly from the notebook outputs.

## 2. Methodology

The pipeline runs as a loop over the frame sequence. At each time  $t$  I compare the current frame  $I_t$  with a reference background  $B_t$  and produce a binary change map  $M_t$ . I then label connected components of  $M_t$  (the blobs), clean small or noisy components, and match the resulting blobs against the previous frame so that detections can be followed over time. The optional persistent tracker extends matching beyond adjacent frames.

### 2.1 Background initialization

I compute the initial background as the pixel wise mean of the first  $N$  frames, assuming they are mostly empty. This is version one of the algorithm in the motion analysis slides:

$$B_0(x, y) = (1 / N) * \sum_{i=0..N-1} I_i(x, y)$$

The default value in my CONFIG is  $N = 5$ . Five frames is enough to average out shot noise without locking the background to a non empty scene.

### 2.2 Threshold strategies

The change map at time  $t$  is

$$M_t(x, y) = 1 \text{ if } |I_t(x, y) - B_t(x, y)| > \tau, 0 \text{ otherwise}$$

I compare two threshold rules.

1. Manual heuristic. The motion analysis slide on the difference histogram says that a good threshold should reflect the amplitude of the quasi Gaussian noise distribution. I estimate  $\sigma_{\text{noise}}$  from the standard deviation of the difference between adjacent empty frames and set  $\tau = \max(\text{CONFIG}['\tau'], k * \sigma_{\text{noise}})$  with  $k = 3$  and a manual floor of 0.10 in  $[0, 1]$  units. The floor protects against unrealistically small thresholds when the first few frames happen to be very quiet.
2. Otsu. From the segmentation lecture, applied to the absolute difference image via `skimage.filters.threshold_otsu`. This is the same statistical discriminant analysis we saw for grey level segmentation.

### 2.3 Background update

Slow scene drift makes a static  $B_0$  unreliable on long sequences. Version two of the slides updates the background with a running average:

$$B_{t+1}(x, y) = (1 - \alpha) * B_t(x, y) + \alpha * I_t(x, y)$$

Version three only updates pixels marked as background in  $M_t$ , so that foreground objects do not contaminate  $B_t$ . Both modes are supported in the implementation through an `only_static` flag.

## 2.4 Connected components and cleaning

I extract connected components with `skimage.measure.label` and `skimage.measure.regionprops`. Each blob carries the descriptors needed for matching: centroid, area, bounding box, mean intensity inside the blob, eccentricity, and aspect ratio. Cleaning has three steps in order: drop components smaller than `min_blob_area`, apply morphological closing with a disk, then drop small components again in case closing left any sliver alone. This ordering is what the lecture sketch notebook does.

## 2.5 Blob matching

I implement two matching strategies, both built on a greedy one to one assignment over a cost matrix.

- Distance based. Cost is Euclidean distance between centroids. Matches farther than `max_dist` are rejected.
- Combined similarity. Cost is  $w\_distance * (d / max\_dist) + w\_area * |\log(area\_t / area\_t1)|$ . The log area ratio is symmetric in the two areas and penalises a match between a tiny noise blob and a large person, even when they happen to be close.

## 3. Implementation overview

All code lives in a single Colab notebook. Every step is exposed through a small function so the pipeline is easy to read and to reuse. The complete list of tunable parameters sits in a single CONFIG dictionary.

| Parameter                       | Value | Meaning                                           |
|---------------------------------|-------|---------------------------------------------------|
| <code>N_init_frames</code>      | 5     | frames averaged to build $B_0$                    |
| <code>tau</code>                | 0.10  | manual floor on the absolute difference threshold |
| <code>k_sigma</code>            | 3.0   | multiplier for the noise std heuristic            |
| <code>alpha</code>              | 0.05  | running average learning rate                     |
| <code>update_only_static</code> | True  | restrict update to pixels with $M_t = 0$          |
| <code>min_blob_area</code>      | 100   | minimum connected component area in pixels        |
| <code>kernel_size</code>        | 3     | disk radius for morphological closing             |
| <code>max_dist</code>           | 60.0  | gating distance for matching, in pixels           |
| <code>w_distance</code>         | 1.0   | weight on normalised distance term                |
| <code>w_area</code>             | 1.0   | weight on log area ratio term                     |
| <code>track_max_gap</code>      | 3     | frames a track may be unobserved before closing   |
| <code>track_max_dist</code>     | 80.0  | gating distance when extending a track            |

## 4. Experiment 1: provided station video

The provided sequence is 71 JPEG frames from frame0250.jpg to frame0320.jpg. The camera is fixed and the scene is mostly empty between events. This is the well behaved case the pipeline is designed for.

### 4.1 Background initialization and noise estimate

Averaging the first five frames gave a background image with a small artefact: a faint walking figure is still visible on the right side of the floor. That means at least one of the five init frames was not perfectly empty. The histogram of inter frame differences is still tightly concentrated near zero, so  $\sigma_{\text{noise}}$  comes out small (0.034 in  $[0, 1]$  units) and the heuristic threshold settles at the manual floor of 0.102. The ghost is too faint to create false positives at that threshold.

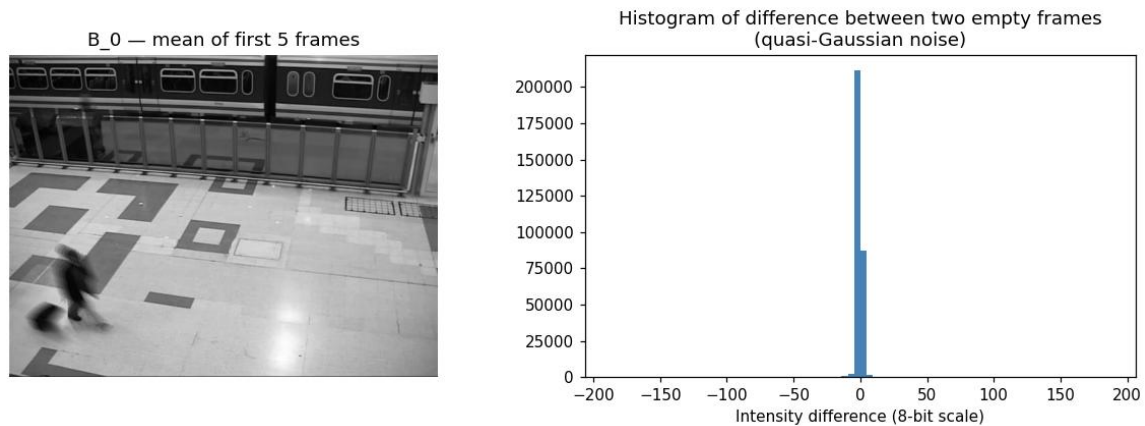


Figure 1.  $B_0$  averaged from the first five frames (left) and the histogram of intensity differences between two empty frames (right). The histogram is concentrated near zero, confirming the quasi Gaussian noise assumption.

### 4.2 Manual tau versus Otsu

On a representative frame with five people walking on the station floor, the manual threshold of 0.102 cleanly extracts the silhouettes. The cleaned map removes salt and pepper noise around the legs and the bounding boxes sit tightly around each person.

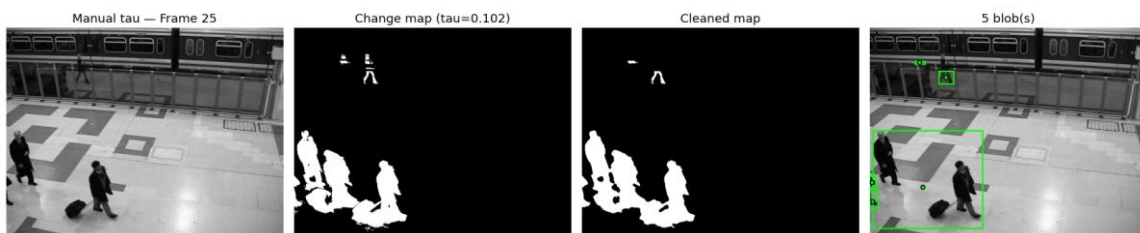


Figure 2. Manual tau on frame 25. From left to right: original frame, raw change map, cleaned map, final overlay with five blob bounding boxes.

Otsu picks a threshold of about 0.24 on the same frame, which is more than twice the manual one. The body interiors of the walking people are stripped out by the higher threshold, and only the high contrast edges survive. The blob count goes up from five to seven not because Otsu finds more people but because each person fragments into smaller pieces.

Frame 25 — manual tau vs Otsu

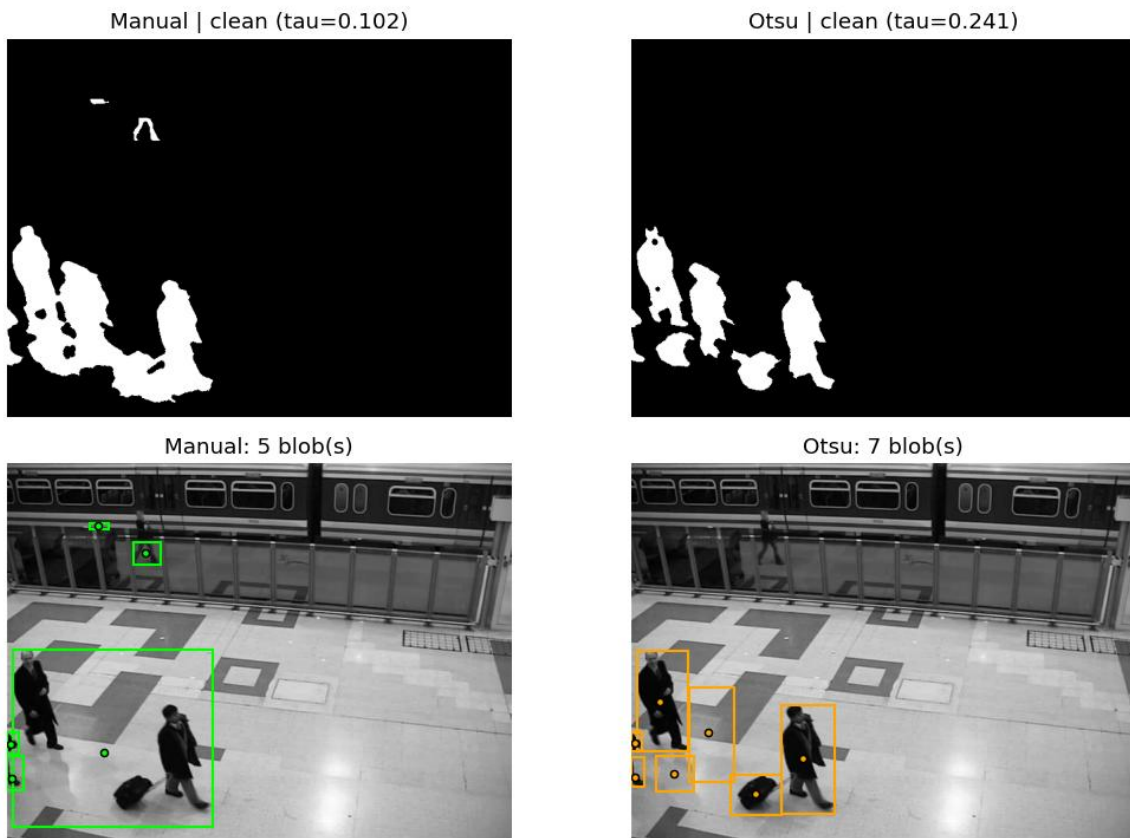


Figure 3. Manual tau (left) and Otsu (right) on frame 25. Otsu over thresholds and fragments the silhouettes.

### Otsu applicability

Otsu assumes a bimodal histogram. On a change detection difference image the foreground class is much smaller than the background class, so Otsu's optimal split lands well inside what is really a Gaussian shaped noise distribution with a long tail. The tail belongs to the foreground, but only its strongest pixels survive the cut. The more empty a frame is, the worse this gets. Two practical fixes are: gate Otsu on a global activity test (run it only when the mean absolute difference exceeds a small noise floor), and lower bound the Otsu output by  $k * \sigma_{\text{noise}}$  so it cannot drop below the noise level. Within those guards Otsu is a useful per frame auto tuning rule. Without them, the manual heuristic threshold is the safer default for this kind of footage.

### 4.3 Static B<sub>0</sub> versus running average

I ran the pipeline twice on the provided video, once with `update_bg=False` and once with `update_bg=True`, and compared the same frame side by side. The static version returns 5 blobs and the running average version returns 6, but the per object quality is clearly different. With the static B<sub>0</sub> the two pedestrians on the left are fused into one large bounding box, and there are two small false positives near the train. With the running average the same two pedestrians are split into two separate boxes, one of the train artefacts disappears, and a small ghost appears on the left edge where stationary foreground is starting to fade into B<sub>t</sub>. The running average therefore trades fewer illumination drift artefacts for a slow leak of static foreground into the background, which is the textbook trade off described in the lecture.



Figure 4. Frame 55 of the provided video. Static  $B_0$  (left) fuses two pedestrians into one blob; running average background (right) splits them and removes one false positive at the cost of a small ghost on the left edge.

#### 4.4 Where the manual pipeline still fails

Two recurring failure modes show up on the station video. First, when a person crosses a partially shadowed region the silhouette breaks into smaller fragments that even morphological closing does not always bridge. A larger kernel size merges them but also fuses neighbouring people. Second, with `update_bg=False` the static background eventually disagrees with the scene as illumination drifts, and false positive regions appear in shadowed corners. Switching `update_bg=True` removes most of that drift, at the cost of slowly absorbing stationary foreground.

### 5. Experiment 2: my custom videos

I recorded three videos with my phone and saved them on Drive. Each video tests a different challenge: a moving camera, a steady camera in a busier scene, and a short low quality clip of the steady scene used as a sanity check.

| File      | Size  | Role                                  | Raw frames | Frames kept | Step |
|-----------|-------|---------------------------------------|------------|-------------|------|
| test1.mp4 | 44 MB | moving camera, full resolution        | 890        | 178         | 5    |
| test2.mp4 | 22 MB | steady camera, busy scene             | 431        | 144         | 3    |
| test3.mp4 | 1 MB  | short low quality clip of test2 scene | 219        | 219         | 1    |

Phone videos record at high frame rate, so before running the pipeline I extract frames with a stride to keep the sequence light. Bigger files get a larger stride so that Colab RAM stays under control. The stride based subsampling does the same job as reducing the source frame rate, without the need to re encode the original files.

#### 5.1 test1.mp4 (moving camera)

The first five frames already cover a different part of the street than later frames, so the noise sigma estimate jumps to about 0.108 and the heuristic clamps tau up to 0.325. Even with that high threshold the pipeline reports between 40 and 70 blobs per frame because almost every pixel of every frame differs from  $B_0$ . The pipeline is doing exactly what static background change detection should do; the problem is that the assumption it relies on (a fixed camera) is violated.



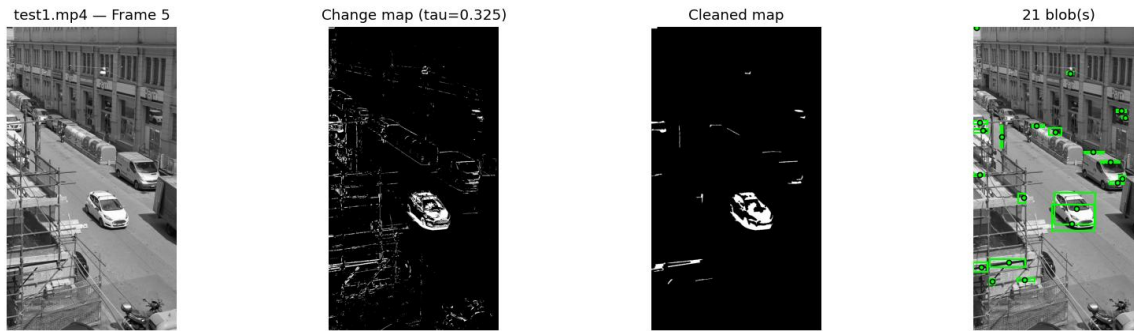


Figure 5. test1.mp4, an early frame. Twenty one bounding boxes are reported, including scaffolding edges, parked cars, and building details that only differ from  $B_0$  because the camera has moved.

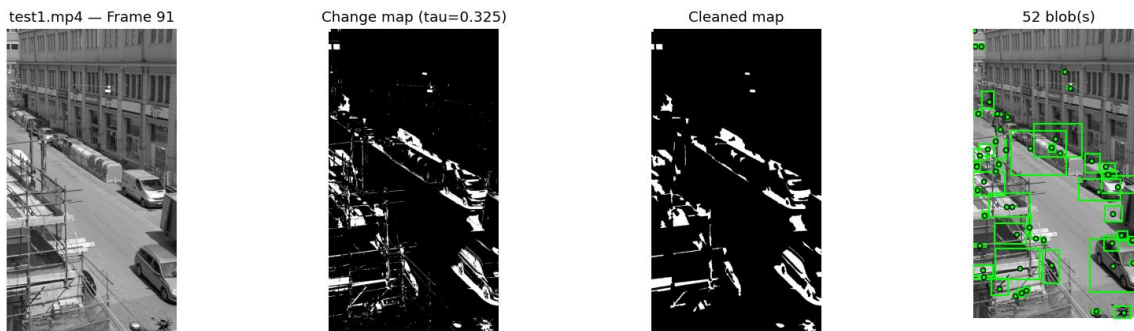


Figure 6. test1.mp4, a later frame from a different camera angle. The change map lights up almost the entire frame because the static  $B_0$  was built from a completely different scene.

## 5.2 test2.mp4 (steady camera, busy scene)

On test2 sigma\_noise is small, the manual floor of  $\tau = 0.100$  wins, and the pipeline tracks the genuine moving object (the silver car around frame 74) cleanly. Some false positives still show up around scaffolding edges and parked motorcycles, especially in busier frames, but the order of magnitude is reasonable: 18 to 46 blobs per frame, dominated by a small number of real objects plus persistent background structure.

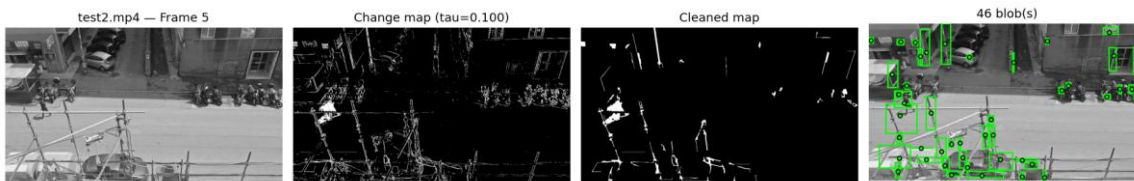


Figure 7. test2.mp4. The silver car on the road is cleanly captured; the scaffolding produces a steady ring of small false positives that survive cleaning because they are near the cleaning area threshold.

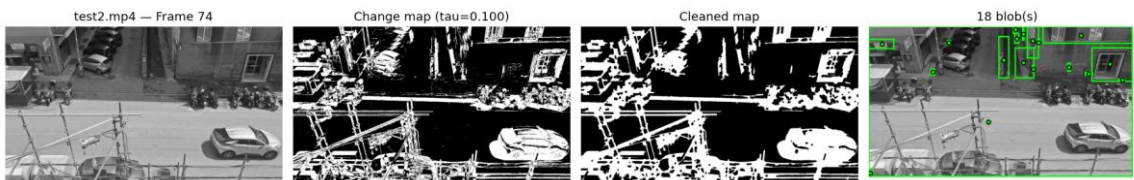


Figure 8. test2.mp4 later in the sequence. The pipeline keeps tracking the real moving objects as the scene evolves, with the running average background absorbing slow exposure drift.

## 5.3 test3.mp4 (short low quality clip)

test3 is a lower resolution version of the same scene as test2. Frame 5 returns zero blobs (matches  $B_0$ ), and the pipeline picks up the moving silver car and the motorcycle as the sequence evolves.

Manual  $\tau = 0.100$  is a good operating point. This is the cleanest of the four cases and shows that the pipeline behaves correctly when the static background assumption holds.

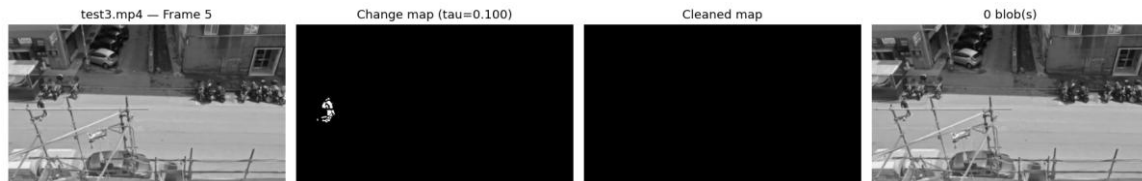


Figure 9. test3.mp4. Frame 58 isolates a motorcycle and a small spurious blob; frame 218 wraps the moving silver car in a tight bounding box.



Figure 10. test3.mp4 near the end. The car is still cleanly tracked and only a few small blobs survive cleaning.

## 5.4 What this experiment shows

The static background change detection assumption breaks the moment the camera moves more than a few pixels. test3 respects the assumption and works; test1 and test2 violate it to different degrees and the pipeline reacts to background structure as if it were foreground. This is consistent with what the lecture warns about. It also means that my three videos taken together form a useful spectrum, from clean to violated, that lets me observe the pipeline's behaviour at different points along it.

## 6. Matching analysis

For every video I compared the two matching strategies (distance only and combined similarity) on every consecutive pair of frames. The match rate is the number of matched pairs divided by the smaller of the two blob counts. A rate of 1.0 means every blob in the smaller frame found a counterpart.

| Video            | Distance | Similarity | Winner                |
|------------------|----------|------------|-----------------------|
| provided station | 0.99     | 0.99       | tie                   |
| test1 (moving)   | 0.88     | 0.88       | similarity, by a hair |
| test2 (steady)   | 0.93     | 0.94       | similarity            |
| test3 (short)    | 1.00     | 1.00       | tie                   |

On well behaved sequences (the provided station video and test3) every blob in frame  $k - 1$  finds its counterpart in frame  $k$ , and the two strategies score essentially perfectly. On test2, where the camera is steady but the scene is busier, the similarity rule edges the distance rule by about one match per hundred. On test1, where the camera moves, both rules drop together to 0.88. The area ratio penalty does not rescue matching when the underlying detections are themselves unstable.

### 6.1 Why distance and similarity often tie

When the cleaner returns clean, well separated blobs that move only a few pixels per frame, the closest blob in the next frame is almost always the right one and the area term changes nothing.



Similarity differs from distance only when two candidate blobs are about equally close but have very different sizes, which is a situation that occurs on test2 (a real moving car next to a small noise fragment from the scaffolding edge) and on the busier frames of the provided station video.

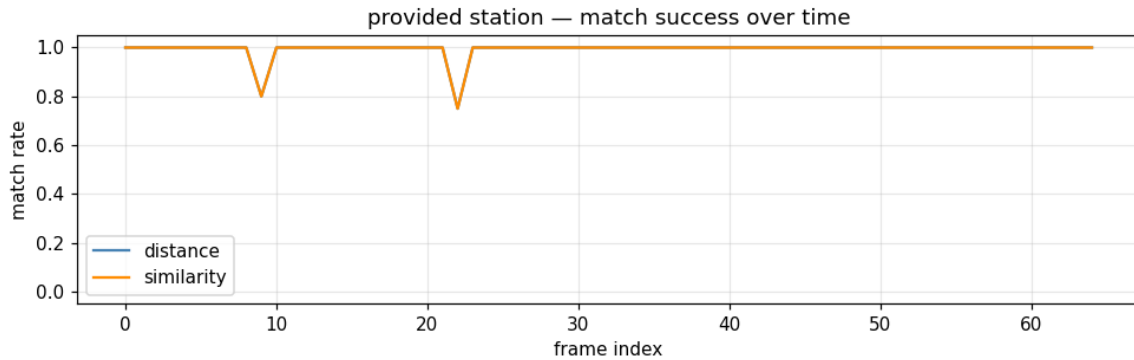


Figure 11. Provided station video. Both strategies stay at or near 1.0 for the whole sequence.

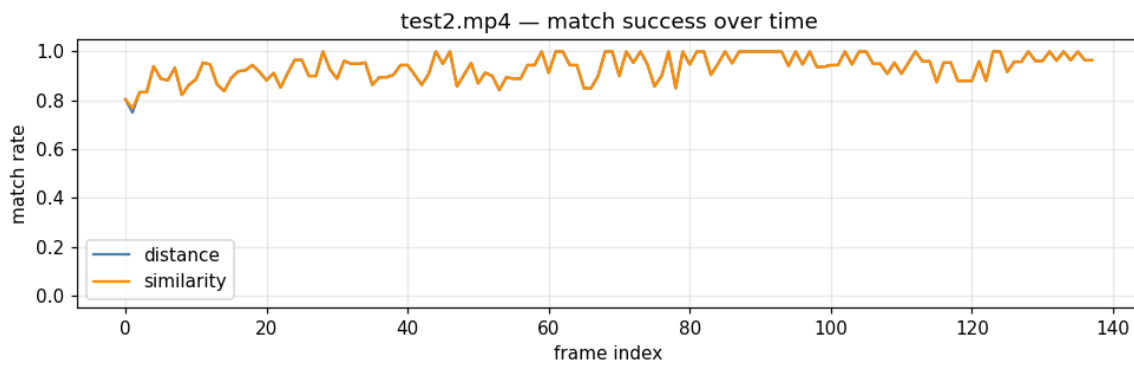


Figure 12. test2.mp4. Both rates oscillate around 0.93 and 0.94. Similarity stays slightly above distance on the busier frames.

## 6.2 Why moving camera footage cannot be saved by either rule

On test1 every frame produces a different population of blobs because the change detection map keeps lighting up new background structure as the camera moves. The matching step is being asked to associate two essentially independent point clouds. Neither distance nor area ratio is going to find correspondences that are not really there. The fix is upstream of the matcher: stabilise the camera, or compensate camera motion before differencing.

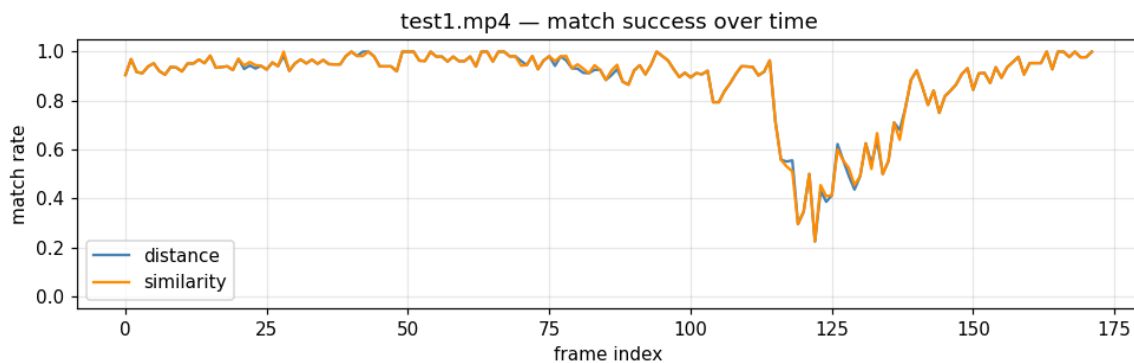


Figure 13. test1.mp4. Both strategies drop to about 0.88, with sharp dips at moments where the camera reframes.

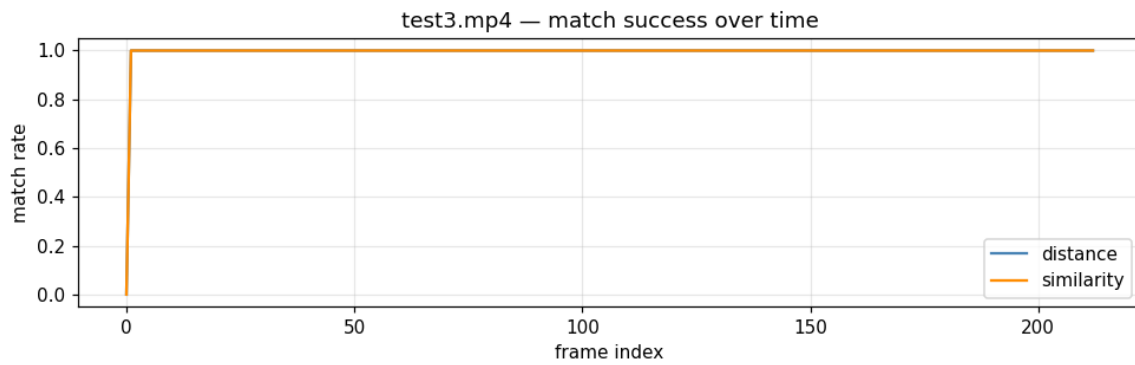


Figure 14. test3.mp4. Both strategies sit at 1.00 across the whole sequence.

### 6.3 Practical takeaway

- For short, low noise sequences with stable framing, the distance rule is sufficient and avoids the extra  $w\_area$  hyperparameter.
- For longer or noisier sequences with consistent object sizes, the similarity rule is the safer default. The cost is one extra parameter; the gain is a small but consistent improvement.
- Neither strategy can rescue change detection output that already contains spurious blobs from camera motion.

## 7. Optional extension: persistent track IDs

The matching step in section 6 only links adjacent frames. The assignment suggests integrating matches over longer time spans as an optional extension. I implemented a small greedy tracker on top of the existing matcher: every detected blob receives a persistent track ID that survives up to `track_max_gap` frames of missed detection. The cost function is the same combined similarity used in section 6 (centroid distance plus log area ratio), and a gap tolerance window provides temporal continuity. Unmatched blobs spawn new tracks, and tracks idle for more than `track_max_gap` frames are closed.

| Video            | Unique tracks | Mean length | Median length | Longest | Tracks > 5 frames |
|------------------|---------------|-------------|---------------|---------|-------------------|
| provided station | 15            | 21.1        | 9.0           | 66      | 10 / 15 (67%)     |
| test1 (moving)   | 654           | 14.5        | 5.0           | 123     | 325 / 654 (50%)   |
| test2 (steady)   | 245           | 14.8        | 3.0           | 139     | 101 / 245 (41%)   |
| test3 (short)    | 20            | 47.6        | 32.5          | 148     | 17 / 20 (85%)     |

### 7.1 The median is the diagnostic

On the provided video (median 9 frames) and on test3 (median 32.5 frames) most tracks live long enough to be useful. On test2 the median collapses to 3 frames, which tells me that the bulk of detections are short bursts of noise that the gap tolerance of 3 frames cannot stitch together. On test1 the count of unique tracks balloons to 654, again confirming the moving camera diagnosis from section 6: every reframe spawns a fresh population of IDs that never get linked back to anything.

provided station. Frame 38, 4 blobs, 15 unique tracks so far



Figure 15. Persistent tracks on the provided station video. Each colour is one track, and the line behind every centroid is a fading trail of past positions.

test3.mp4. Frame 112, 3 blobs, 20 unique tracks so far

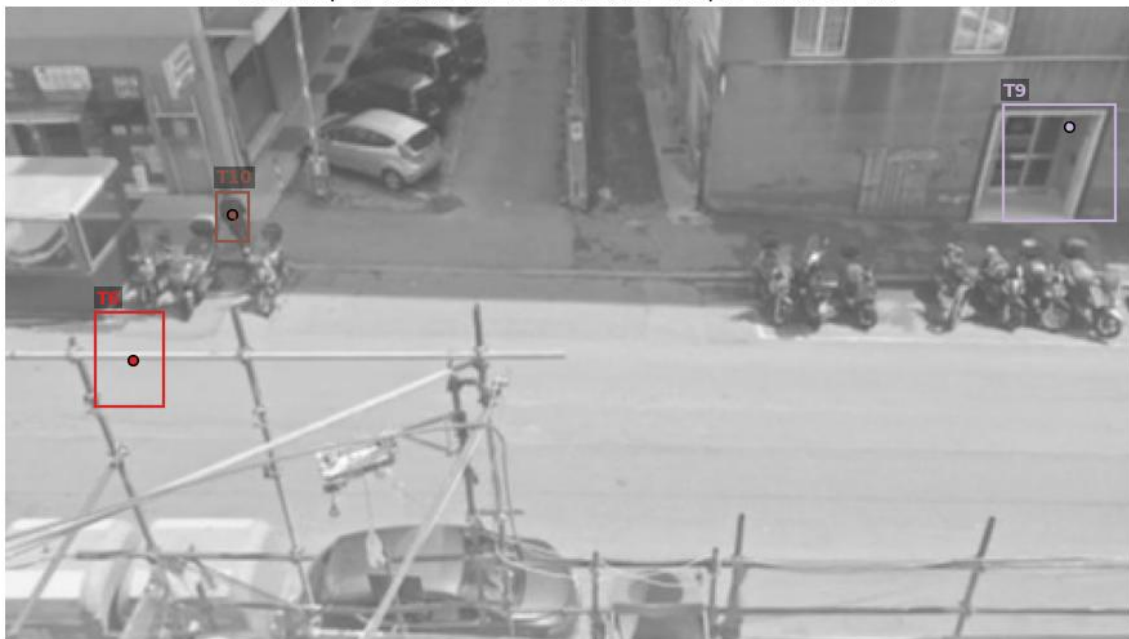


Figure 16. Persistent tracks on test3.mp4. The moving silver car holds a single ID for most of the sequence.

## 7.2 The longest track is reassuring on every video

Even on test1 the tracker manages to hold one object across 123 frames. On test2 the longest track covers all 139 processed frames, and on test3 it covers 148 of 214. So the tracker can hold onto an object for the whole clip when the underlying detections are stable enough. On test1 the long track is more likely to be a piece of background structure that stays inside the frame across many camera moves, which is interesting because it shows the same matching mechanism applied to the wrong target.

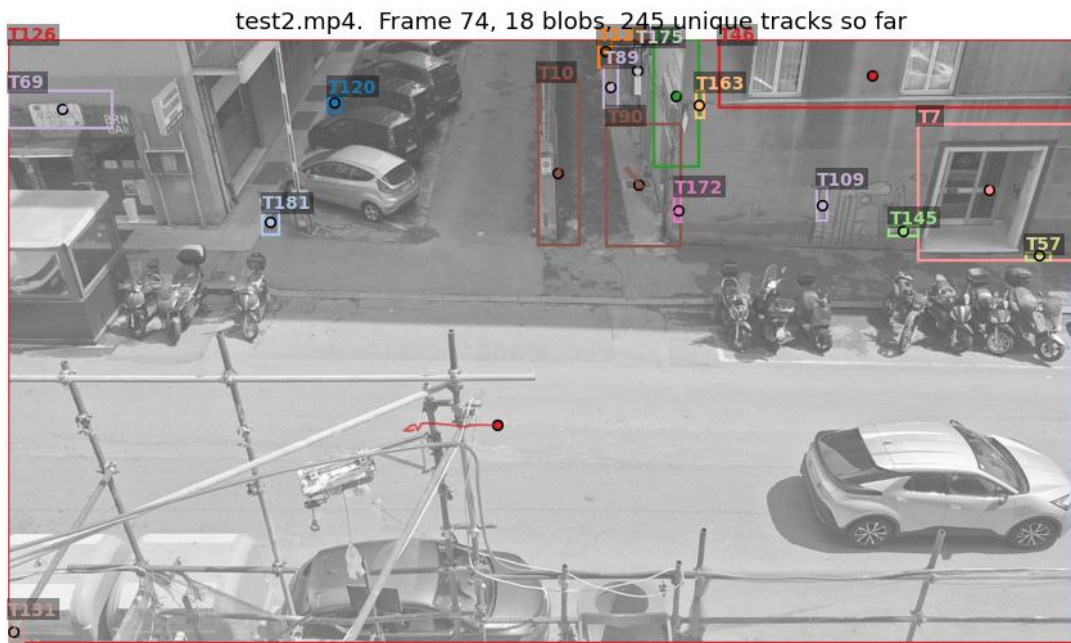


Figure 17. Persistent tracks on test2.mp4. Real moving objects get long stable IDs while small noise blobs collapse into many short tracks.

test1.mp4. Frame 91, 52 blobs, 654 unique tracks so far



Figure 18. Persistent tracks on test1.mp4. The number of unique IDs explodes because every camera move creates a fresh population of blobs.

### 7.3 Parameter sensitivity

The two knobs are `max_gap` (how many frames a track may disappear before it is closed) and `max_dist` (how far the next detection may be from the last one). With `max_gap` = 3 and `max_dist` = 80 the tracker forgives brief mis detections without linking unrelated objects. Raising `max_gap` chains noise blobs into longer spurious tracks. Lowering it splits real trajectories. Raising `max_dist` lets the tracker associate detections across larger jumps but at the cost of linking different objects when motion is fast.

### 7.4 Limitations and possible upgrades

This tracker is greedy and reactive. Two upgrades would make it stronger: a Hungarian assignment instead of greedy (better when many candidates compete for the same target), and a constant velocity Kalman prediction inside the cost so the matcher reasons about predicted positions rather than last observed ones. Either change plugs into the existing function signature without affecting the rest of the notebook.

## 8. Conclusions

Working through this assignment, what stood out to me is how much the static background assumption shapes everything downstream. When the assumption holds the pipeline is almost trivial; when it fails, no amount of tuning in the matcher or in the tracker can compensate.

1. The static background assumption is fragile. The provided video and test3 respect it; their per frame match rates are 0.99 to 1.00 and the tracker recovers a small, stable set of long lived trajectories (15 and 20 unique tracks, longest 66 and 148 frames). test1 violates the assumption because the camera moves; the match rate drops to 0.88 and the tracker spawns 654 unique IDs because every reframe creates a fresh population of blobs. The fix lives upstream of the matcher.
2. Otsu does not transfer cleanly from segmentation to change detection. The difference image is rarely bimodal because the foreground class is much smaller than the background class. Otsu's threshold (around 0.24 on the provided video) lands well above the noise floor and inside the foreground, fragmenting silhouettes and inflating the blob count. Two practical fixes are a noise floor activity test before running Otsu, and a sigma based lower bound on its output.
3. Distance and similarity matching tie when detections are clean and inter frame motion is small (1.00 on test3, 0.99 on the provided station). They diverge slightly when noise blobs sit near a real object: similarity edges distance on test2 (0.94 versus 0.93). On test1 they are tied at 0.88 because the area ratio cannot rescue matching when the blob population itself is unstable.
4. Persistent tracking turns per frame matches into trajectories with stable IDs. The median track length is the cleanest summary of detection quality: 9 frames on the provided video, 32.5 on test3, 5 on test1, 3 on test2. A short median is the tracker reporting that single frame detection is not yet good enough.
5. The running average background helps with slow drift and gives tighter per object detections, at the cost of a small ghost where stationary foreground starts fading into B<sub>t</sub>. On a frame where the static B<sub>0</sub> fuses two adjacent walkers into one large blob, the running average splits them into two separate blobs.

### What I would try next

- A motion compensation step (frame to frame registration) before the difference, so the pipeline survives a moving camera.
- Hungarian assignment with a Kalman based predicted position cost in the tracker, building on the implementation in section 7.
- A simple shadow suppression test in HSV colour space, since shadows triggered some of the test2 false positives.

## Appendix A. Files included with the submission

- Mojtaba\_Alehosseini\_5965588\_midterm.ipynb. The full Colab notebook, run end to end with all outputs preserved.
- test1.mp4, test2.mp4, test3.mp4. The three videos I recorded and analysed in section 5.
- Mojtaba\_Alehosseini\_5965588\_midterm\_report.pdf. This report.



## Appendix B. How to reproduce

1. Open `Mojtaba_Alehosseini_5965588_midterm.ipynb` in Google Colab.
2. Run cells top to bottom. Sections 0 to 8 use the provided station video and require Drive access at the path set in cell 5.
3. Section 9 reads the three custom videos from the same Drive folder. The frame stride per video is set in the `CUSTOM_VIDEOS` list inside that section.
4. Sections 10 and 11 run automatically once the previous sections have produced `result_manual` and `custom_results`.